Analyse von **Apache Log4j2** mit **Joern** und **Semgrep**

Alexander Ley und Jonathan Ndinga

11. Juni 2026

Projekt Überblick von **Apache Log4j2**



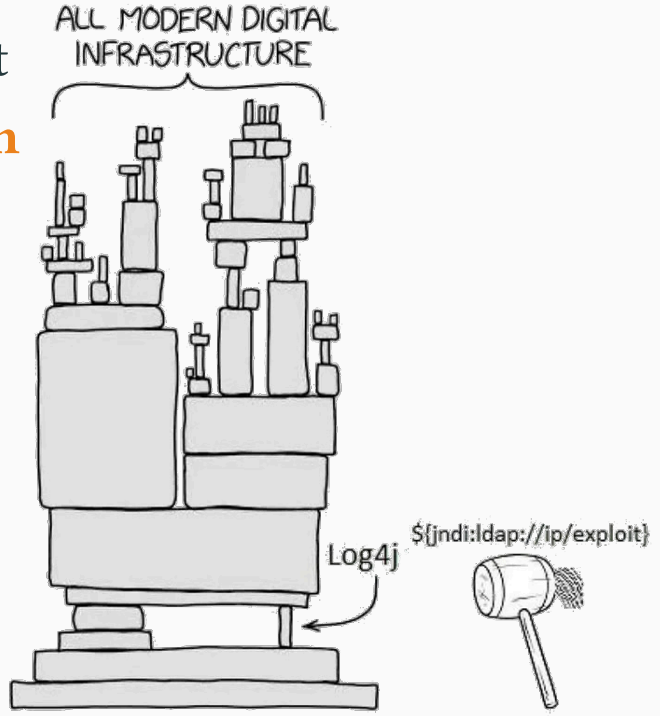
- Standard Logging Framework für `Java` in der Industrie
  - gilt als Vorreiter für andere Logging-Frameworks
- Etliche Adaptionen für andere Programmiersprachen
- Auf möglichst hohe Geschwindigkeit ausgelegt, um die Applikation möglichst nicht zu beinträchtigen
- Vollständige Neuentwicklung im Jahre 2014 als Antwort auf die Schwachstellen in **Log4j** → **Log4j2**
- bekanntwerden der **Log4Shell** Sicherheitslücke im Dezember 2021

Zahlen und Fakten:

Entwickler: Apache Software Foundation	**Gegründet**: 1996
Programmiersprache: Java	**Vollständie Neuentwicklung**: Juli 2015
Commits: 14.664	**Aktuelle Version**: 2.26.0
Java Code Zeilen: 348.872	**Forks**: 1.700

“Log4shell” Sicherheitslücke *(CVE-2021-44228)* in Log4j2

- Wurde im Dezember 2021 in der aktuellen **Log4j2** Version bekannt
  - ermöglicht **Remote Code Execution**
  - CVSS von 10
  - gilt als eine der gefährlichsten Sicherheitslücken überhaupt
    - **große Reichweite** mit potentiell **verheerenden Folgen**
- viele bekannte Softwareprodukte waren betroffen:
  - Cloudflare, iCloud, Minecraft: Java Edition, …

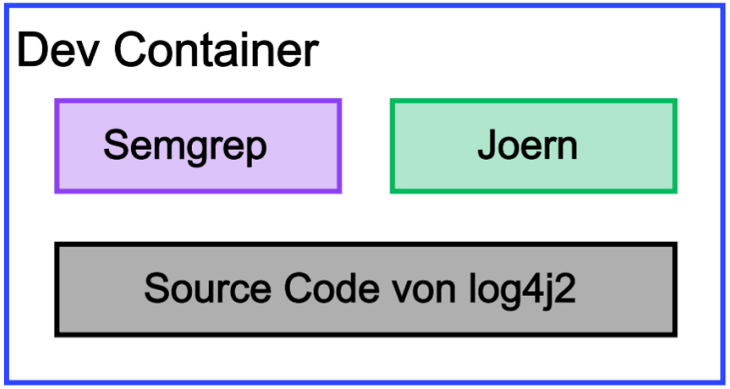


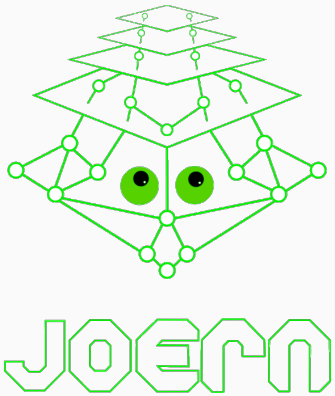
Ist die Sicherheitslücke nun geschlossen?

- **Ja und Nein**: Veröffentlichung eines Patches, aber nicht alle Softwareprodukte updaten Log4j
  - Somit ist diese Sicherheitslücke potentiell noch immer relevant

Tool Setup mit Joern und Semgrep

- Verwendung des in der Übung bereitgestellten DevContainer wo **Joern** und **Semgrep** bereits installiert waren
- Clonen des Source Code von **Apache Log4j2**[1]



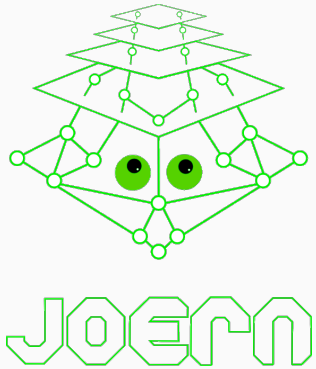


Semgrep

[1] https://github.com/apache/logging-log4j2

1. Anzahl der Methoden

`cpg.method.l.size` $\Rightarrow$ 19.643



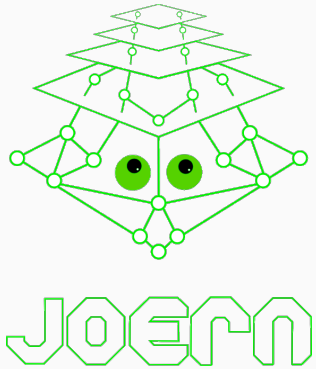
Joern: Erste Experimente

1. Anzahl der Methoden

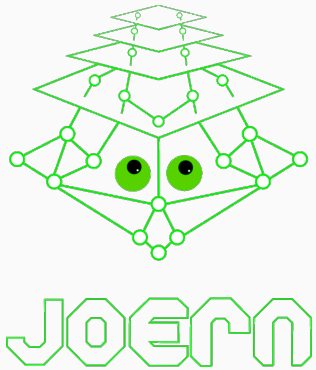
`cpg.method.l.size` $\Rightarrow$ 19.643

2. Anzahl der Methodenaufrufe

`cpg.call.l.size` $\Rightarrow$ 110.315



Joern: Erste Experimente



1. Anzahl der Methoden

```
cpg.method.l.size
```

$\Rightarrow$ 19.643

2. Anzahl der Methodenaufrufe

```
cpg.call.l.size
```

$\Rightarrow$ 110.315

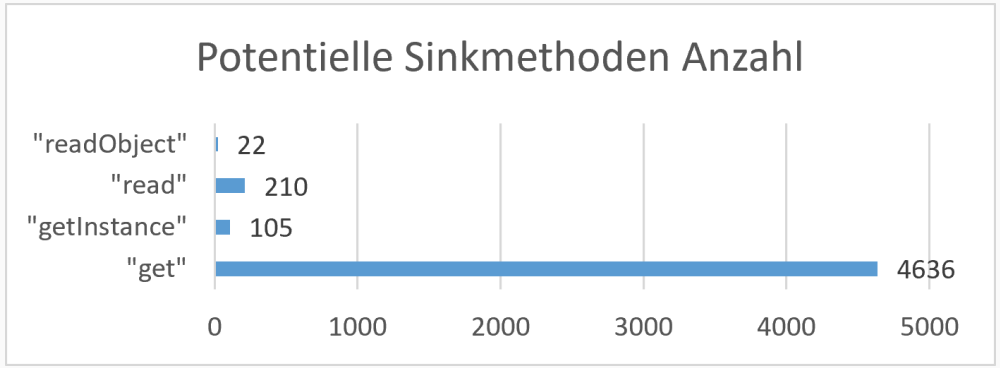
3. Anzahl der Methodenaufrufe die "execute" enthalten

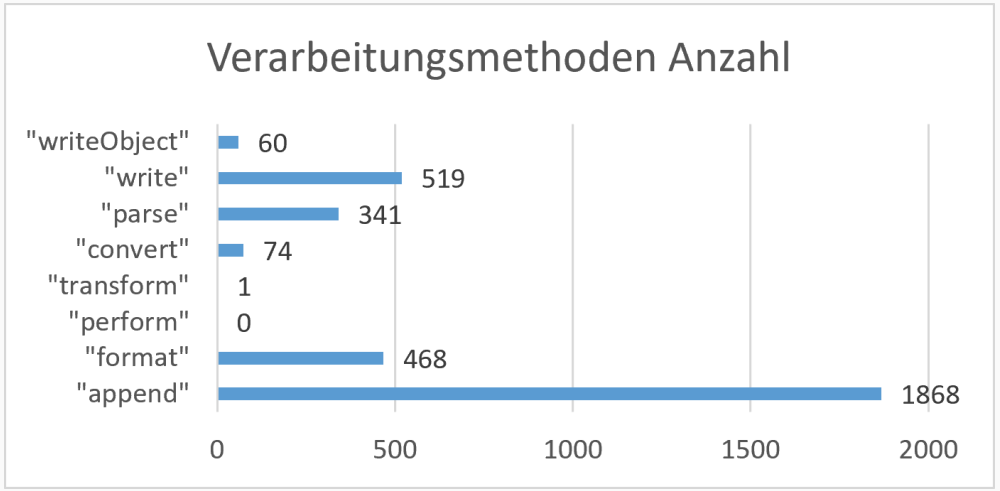
```
cpg.call.filter(_.name.toLowerCase().contains("execute")).l.size
```

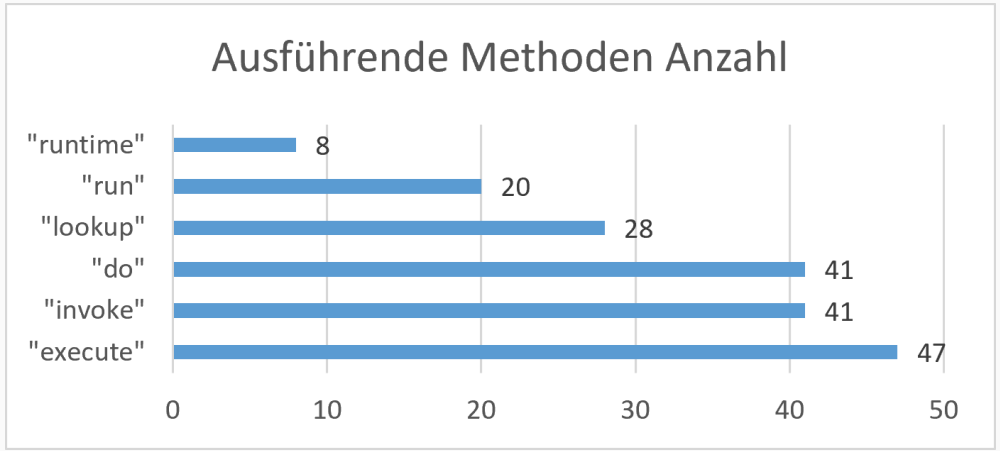
$\Rightarrow$ 47

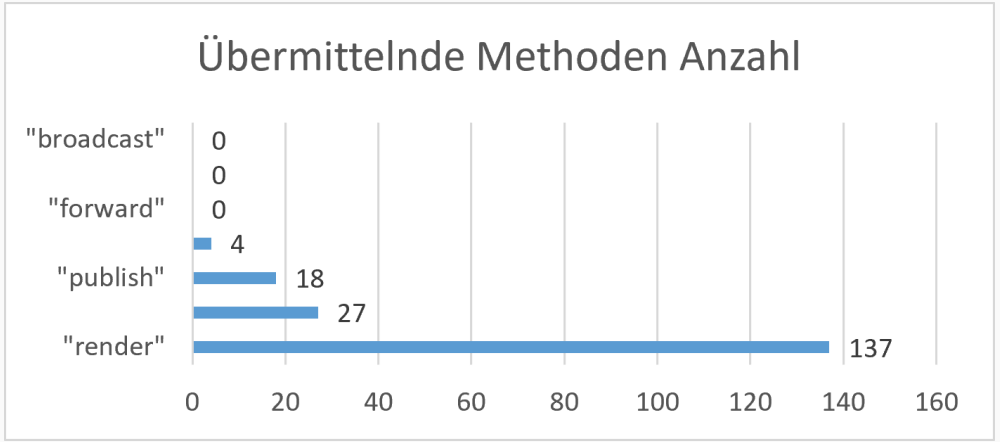
$\Rightarrow$ "execute"-Methoden als potentielle **Sinks**

Joern: Weitere Methoden









Joern: Ein Ausschnitt der “execute”-Methoden

- `execute()`
- `executeScript()`
- `executeBatch()`
- `executeUpdate()`
- `executeUpdate(String s)`
- `execute(Runnable runnable)`
- `execute(boolean endOfBatch)`
- `execute(CommandLine parsed)`
- `execute(FileVisitor visitor)`
- `execute(Bindings bindings)`
- `execute(String id, final Bindings bindings)`
- `execute(String name, File source, File destination, boolean deleteSource)`
- `execute(File source, File destination, boolean deleteSource)`
- `execute(File source, File destination, boolean renameEmptyFiles)`
- `execute(File source, File destination, boolean deleteSource, int level)`

Joern: Ein Ausschnitt der “execute”-Methoden

- `execute()`
- `executeScript()`
- `executeBatch()`
- `executeUpdate()`
- `executeUpdate(String s)`
- `execute(Runnable runnable)`
- `execute(boolean endOfBatch)`
- `execute(CommandLine parsed)`
- `execute(FileVisitor visitor)`
- `execute(Bindings bindings)`
- `execute(String id, final Bindings bindings)`
- `execute(String name, File source, File destination, boolean deleteSource)`
- `execute(File source, File destination, boolean deleteSource)`
- `execute(File source, File destination, boolean renameEmptyFiles)`
- `execute(File source, File destination, boolean deleteSource, int level)`

$\Rightarrow$ Potentielle **Command Injections**

Joern: Ein Ausschnitt der “execute”-Methoden mit Klassen

- `execute()` in `org.apache.logging.log4j.core.appender.rolling.action.Action`
- `executeScript()` in [...] `.log4j.core.appender.rolling.action.DeleteAction`
- `executeBatch()` in `java.sql.PreparedStatement`
- `executeUpdate()` in `java.sql.PreparedStatement`
- `executeUpdate(String s)` `java.sql.Statement`
- `execute(Runnable runnable)` `java.util.concurrent.ExecutorService`
- `execute(boolean endOfBatch)` in [...] `.core.async.RingBufferLogEvent`
- `execute(CommandLine parsed)` in [...] `.core.tools.picocli.CommandLine`
- `execute(FileVisitor visitor)` in [...] `.core.appender.rolling.action.AbstractPathAction`
- `execute(Bindings bindings)` in [...] `.core.script.ScriptManager§ScriptRunner`

Joern: Ein Ausschnitt der “execute”-Methoden mit Klassen

- `execute()` in `org.apache.logging.log4j.core.appender.rolling.action.Action`
- `executeScript()` in [...] `.log4j.core.appender.rolling.action.DeleteAction`
- `executeBatch()` in `java.sql.PreparedStatement`
- `executeUpdate()` in `java.sql.PreparedStatement`
- `executeUpdate(String s)` `java.sql.Statement`
- `execute(Runnable runnable)` `java.util.concurrent.ExecutorService`
- `execute(boolean endOfBatch)` in [...] `.core.async.RingBufferLogEvent`
- `execute(CommandLine parsed)` in [...] `.core.tools.picocli.CommandLine`
- `execute(FileVisitor visitor)` in [...] `.core.appender.rolling.action.AbstractPathAction`
- `execute(Bindings bindings)` in [...] `.core.script.ScriptManager§ScriptRunner`

⇒ Potentielle **SQL Injections**

Joern: SQL-Injection: Taint Analyse

Taint Methoden

- **Sources**: Die “`get`”-Methoden
- **Sinks**: “`executeBatch`” und “`executeUpdate`”
- **Sanitizer**: “`prepareStatement`” und “`replaceAll`”

Joern: SQL-Injection: Taint Analyse

Taint Methoden

- **Sources**: Die “`get`”-Methoden
- **Sinks**: “`executeBatch`” und “`executeUpdate`”
- **Sanitizer**: “`prepareStatement`” und “`replaceAll`”

Flows

- 22 flows ohne Sanitizer
- 14 flows mit Sanitizer
  - 4 flows ohne `statement.clearParameters()`

Joern: SQL-Injection: Taint Analyse

Taint Methoden

- **Sources**: Die “`get`”-Methoden
- **Sinks**: “`executeBatch`” und “`executeUpdate`”
- **Sanitizer**: “`prepareStatement`” und “`replaceAll`”

Flows

- 22 flows ohne Sanitizer
- 14 flows mit Sanitizer
  - 4 flows ohne `statement.clearParameters()`

Ergebnisse

- 2 True Positive (TP)
  (also für weitere Analysen)
- 2 False Negatives (FP)

Joern: SQL-Injection: Anmerkungen

- Es gibt nur einen Ort, wo SQL Anfragen stattfinden: `JdbcDatabaseManager.java`
  - alle flows laufen über den selben Code

Joern: SQL-Injection: Anmerkungen

- Es gibt nur einen Ort, wo SQL Anfragen stattfinden: `JdbcDatabaseManager.java`
  - alle flows laufen über den selben Code
- Verwendung von der Klasse `PreparedStatement`

Joern: SQL-Injection: Anmerkungen

- Es gibt nur einen Ort, wo SQL Anfragen stattfinden: `JdbcDatabaseManager.java`
  - alle flows laufen über den selben Code
- Verwendung von der Klasse `PreparedStatement`

Persönliche Einschätzung

$\Rightarrow$ **SQL-Injection eher unwahrscheinlich**

Joern: Command-Injection: Taint Analyse

- Es wurde nirgendwo die Klasse `ProcessBuilder` verwendet !

Joern: Command-Injection: Taint Analyse

- **Sources**: Die “`get`”-Methoden
- **Sinks**: “`execute`” und “`executeScript`”
- **Sanitizer**:
  1. Alles wo Eingaben konvertiert werden
  2. Eingaben, die offensichtlich unbedenklich waren

Joern: Command-Injection: Taint Analyse

- **Sources**: Die “`get`”-Methoden
- **Sinks**: “`execute`” und “`executeScript`”
- **Sanitizer**:
  1. Alles wo Eingaben konvertiert werden
  2. Eingaben, die offensichtlich unbedenklich waren

Flows:

- 150 flows ohne Sanitizer
- 38 flows mit Sanitizer

Joern: Command-Injection: Taint Analyse

- **Sources**: Die “`get`”-Methoden
- **Sinks**: “`execute`” und “`executeScript`”
- **Sanitizer**:
  1. Alles wo Eingaben konvertiert werden
  2. Eingaben, die offensichtlich unbedenklich waren

Flows:

- 150 flows ohne Sanitizer
- 38 flows mit Sanitizer

Ergebnisse

- 18 True Positive (TP)
  (alle sehr ähnlich)
- 20 False Negatives (FP)

Joern: Command-Injection: Anmerkungen

- TP waren vielmals ähnlich
- Es wurden Dateinamen und Scripte als Hauptquellen identifiziert
- viele sehr lange flows
- 2 flows schienen sehr kurz

Joern: Command-Injection: Anmerkungen

- TP waren vielmals ähnlich
- Es wurden Dateinamen und Scripte als Hauptquellen identifiziert
- viele sehr lange flows
- 2 flows schienen sehr kurz

Beispiel

nodeType	tracked	line	method	file
Call	`parsed.getCommand()`	455	execute	`log4j-core/src/main/java/org/apache/logging/log4j/core/tools/picocli/CommandLine.java`
Identifier	`Object command = parsed.getCommand()`	455	execute	[...]/CommandLine.java
Identifier	`command instanceof Runnable`	456	execute	[...]/CommandLine.java
Identifier	`command instanceof Callable`	463	execute	[...]/CommandLine.java
Call	`(Callable<Object>) command.call()`	465	execute	[...]/CommandLine.java
Return	`return ((Callable<Object>) command).call()`	465	execute	[...]/CommandLine.java
MethodReturn	RET	454	execute	[...]/CommandLine.java
Call	`execute(parsedCommands.get(0))`	500	handleParseResult	[...]/CommandLine.java

Joern: Command-Injection: Anmerkungen

- TP waren vielmals ähnlich
- Es wurden Dateinamen und Scripte als Hauptquellen identifiziert
- viele sehr lange flows
- 2 flows schienen sehr kurz

Beispiel

nodeType	tracked	line	method	file
Call	`parsed.getCommand()`	455	execute	`log4j-core/src/main/java/org/apache/logging/log4j/core/tools/picocli/CommandLine.java`
Identifier	`Object command = parsed.getCommand()`	455	execute	[...]/CommandLine.java
Identifier	`command instanceof Runnable`	456	execute	[...]/CommandLine.java
Identifier	`command instanceof Callable`	463	execute	[...]/CommandLine.java
Call	`(Callable<Object>) command.call()`	465	execute	[...]/CommandLine.java
Return	`return ((Callable<Object>) command).call()`	465	execute	[...]/CommandLine.java
MethodReturn	RET	454	execute	[...]/CommandLine.java
Call	`execute(parsedCommands.get(0))`	500	handleParseResult	[...]/CommandLine.java

⇒ flows sind deutlich länger

Joern: Command-Injection: `CommandLine` Klasse

- die Klasse `CommandLine` ist ein **Command-Line-Parser** und kann Commands ausführen
  - sie bekommt ein Objekt, dass einen Command wiederspiegeln soll

Joern: Command-Injection: `CommandLine` Klasse

- die Klasse `CommandLine` ist ein **Command-Line-Parser** und kann Commands ausführen
  - sie bekommt ein Objekt, dass einen Command wiederspiegeln soll

Beispiel für einen `encrypt` command

```
@Command(header = "Encrypt FILE(s), or standard input, to standard output or to the output file.", version = "v1.2.3")
public class Encrypt {

    @Parameters(type = File.class, description = "Any number of input files")
    private List<File> files = new ArrayList<File>();

    @Option(names = { "-o", "--out" }, description = "Output file (default: print to console)")
    private File outputFile;

    @Option(names = { "-v", "--verbose"}, description = "Verbosely list files processed")
    private boolean verbose;

    @Option(names = { "-h", "--help", "-?", "-help"}, usageHelp = true, description = "Display this help and exit")
    private boolean help;

    @Option(names = { "-V", "--version"}, versionHelp = true, description = "Display version info and exit")
    private boolean versionHelp;
}
```

Joern: Command-Injection: `CommandLine` Klasse

- die Klasse `CommandLine` ist ein **Command-Line-Parser** und kann Commands ausführen
  - sie bekommt ein Objekt, dass einen Command wiederspiegeln soll
  - sie befüllt es mit den Werten die **untrusted** von der Kommandozeile als `String` eingelesen werden

Joern: Command-Injection: `CommandLine` Klasse

- die Klasse `CommandLine` ist ein **Command-Line-Parser** und kann Commands ausführen
  - sie bekommt ein Objekt, dass einen Command wiederspiegeln soll
  - sie befüllt es mit den Werten die **untrusted** von der Kommandozeile als `String` eingelesen werden

Beispiel

```
-v -o outfile.txt in1 in2
```

```java
@Command(header = "Encrypt FILE(s), or standard input, to standard output or to the output file.", version = "v1.2.3")
public class Encrypt {

    @Parameters(type = File.class, description = "Any number of input files")
    private List<File> files = new ArrayList<File>();

    @Option(names = { "-o", "--out" }, description = "Output file (default: print to console)")
    private File outputFile;

    @Option(names = { "-v", "--verbose"}, description = "Verbosely list files processed")
    private boolean verbose;

    @Option(names = { "-h", "--help", "-?", "-help"}, usageHelp = true, description = "Display this help and exit")
    private boolean help;

    @Option(names = { "-V", "--version"}, versionHelp = true, description = "Display version info and exit")
    private boolean versionHelp;
}
```

Joern: Command-Injection: `CommandLine` Klasse

- die Klasse `CommandLine` ist ein **Command-Line-Parser** und kann Commands ausführen
  - sie bekommt ein Objekt, dass einen Command wiederspiegeln soll
  - sie befüllt es mit den Werten die **untrusted** von der Kommandozeile als `String` eingelesen werden

Sicherheitsprobleme

- Übergebene Argumente werden **nicht ungeprüft**
  - problematische Eingaben bei `File` und `Path` möglich

Beispiel

```
--file ../../../../etc/passwd
--output /tmp/evil
--config C:\Windows\System32\drivers\etc\hosts
```

Joern: Command-Injection: `CommandLine` Klasse

- die Klasse `CommandLine` ist ein **Command-Line-Parser** und kann Commands ausführen
  - sie bekommt ein Objekt, dass einen Command wiederspiegeln soll
  - sie befüllt es mit den Werten die **untrusted** von der Kommandozeile als `String` eingelesen werden

Sicherheitsprobleme

- Übergebene Argumente werden **nicht ungeprüft**
  - problematische Eingaben bei `File` und `Path` möglich
  - die Klasse packt bei Fehlermeldungen die Argumente direkt in die Fehlermeldungen
    $\Rightarrow$ potentielles leaken von sensiblen Daten wie Tokens, Passwörter usw...

Joern: Command-Injection: `CommandLine` Klasse

- die Klasse `CommandLine` ist ein **Command-Line-Parser** und kann Commands ausführen
  - sie bekommt ein Objekt, dass einen Command wiederspiegeln soll
  - sie befüllt es mit den Werten die **untrusted** von der Kommandozeile als `String` eingelesen werden

Sicherheitsprobleme

- Übergebene Argumente werden **nicht ungeprüft**
  - problematische Eingaben bei `File` und `Path` möglich
  - die Klasse packt bei Fehlermeldungen die Argumente direkt in die Fehlermeldungen
    $\Rightarrow$ potentielles leaken von sensiblen Daten wie Tokens, Passwörter usw...

Persönliche Einschätzung

$\Rightarrow$ Diese Klasse würde ich als **sicherheitsrelevante Input-Grenze** oder **potentielles Evil Gadget** markieren

Joern: Meine Erfahrungen

- Ist eher ein Tool um Code besser zu verstehen und zu analysieren
- Ich hätte mir eine Entwicklungsumgebung gewünscht Joern Skripte zu schreiben mit Autovervollständigung usw...
  - sehr umständlich
- Fokus liegt auf der Benutzung der Joern Shell

Community-Regelwerke: `p/java` und `p/owasp`

```
semgrep --config p/java --config p/owasp-top-ten \
  --json --output semgrep_full.json .
```

Tooling-Befund: Der in der Aufgabe vorgegebene Pack `p/owasp` liefert **HTTP 404** er existiert nicht mehr in der Registry. Ersetzt durch `p/owasp-top-ten` (aktuelles OWASP-Äquivalent). Dokumentiert als Setup-Erfahrung.

Scan-Ergebnisse

- **4539** Dateien gescannt (gesamtes Repo)
- **13 Sek** Dauer · **0** Fehler
- **155** Regeln ausgeführt
- **5** Findings gesamt
- **3 ERROR** · **2 WARNING** · **0 INFO**

Top Regel-IDs	#
`...xxe.doctype-decl-missing`	2
`...run-shell-injection`	1
`...unencrypted-socket`	1
`...use-of-md5`	1

Scope: Gesamtes Repo passend zur Joern-Analyse. Ausschluss der Test-/Fuzz-Module via `--exclude` änderte die 5 Findings nicht: alle in Produktions- bzw. Infrastruktur-Code.

Was wurde mit **Semgrep** gefunden?

Datei	Z.	Regel	Schwere	Bew.	Begründung
`log4j-1.2-api/…XmlConfiguration.java`	167	`...xxe.doctype-decl-missing`	ERROR	**TP**	Keine XXE-Härtung: `setValidating(true)`, kein `disableDtdProcessing()`. Nur `Log4jEntityResolver`, der externe Entities **nicht** abschaltet echte XXE-Angriffsfläche im Legacy-1.x-Konfig-Parser
`log4j-core/…XmlConfiguration.java`	188	`...xxe.doctype-decl-missing`	ERROR	**FP**	Dieselbe Regel, aber gehärtet: `disableDtdProcessing()` deaktiviert externe Entities + DTD. Parst nur die vertrauenswürdige eigene Konfig-Datei kein externer Input
`deploy-site.yaml`	77	`...run-shell-injection`	ERROR	**FP**	`${{ github.ref_name }}` fließt per `sed` in `run:`. Job durch `if`-Guard abgesichert (`startsWith('release/')` + Repo-Check), kein `pull_request_target` Ref nicht angreiferkontrolliert
`TcpSocketManager.java`	464	`...unencrypted-socket`	WARNING	**FP**	Plain-TCP-Appender ist by-design; separater `SslSocketManager` für Verschlüsselung. Klartext ist wählbare Option, kein Defekt
`NameUtil.java`	59	`...use-of-md5`	WARNING	**FP**	MD5 nur für eindeutige Konfig-Element-IDs, nicht für Sicherheit; per Javadoc + `@SuppressFBWarnings` bestätigt, `@Deprecated`

Ergebnis: 5 Findings · **1 True Positive** · **4 False Positives**

Triage-Highlight: Zwei XXE-Findings, ein Urteil auseinander

Dieselbe Regel `documentbuilderfactory-disallow-doctype-decl-missing` feuert an zwei Stellen die Bewertung kippt allein am Vorhandensein der Härtung.

True Positive
`log4j-1.2-api/…XmlConfiguration.java:167`

- `setValidating(true)` — DTD-Verarbeitung **aktiv**
- **kein** `disableDtdProcessing()`
- `Log4jEntityResolver` schränkt nur die Log4j-DTD ein, schaltet externe Entities **nicht** ab
- parst Legacy-1.x-Konfig

⇒ echte XXE-Angriffsfläche

False Positive
`log4j-core/…XmlConfiguration.java:188`

- `disableDtdProcessing()` **aktiv**
- externe general- & parameter-Entities deaktiviert
- externes DTD-Laden deaktiviert
- parst nur vertrauenswürdige eigene Konfig

⇒ Regel-Treffer, aber gehärtet

Lehre: Die Regel matcht ein fehlendes Feature-Flag, „sieht“ aber nicht die **äquivalente** Härtung über andere Features. Erst das manuelle Triage trennt echten Fund von Fehlalarm.

Semgrep: Eigener Ansatz Log4j-spezifische Regeln

Motivation: Die Community-Packs lieferten **0** Treffer für die JNDI-Lookup-Klasse — ausgerechnet die Log4Shell-Schwachstellenklasse. Daher zwei eigene Regeln in `custom_rules.yaml`.

Regel 1 — `log4j-jndi-lookup-sink`

```yaml
patterns:
  - pattern: $CTX.lookup($NAME)
  - metavariable-regex:
      metavariable: $NAME
      regex: '^(?!\s*").+'
```

Matcht JNDI-Lookups mit **nicht-konstantem** Argument → Log4Shell-Sink. Bewusst breit (Sink-Discovery), manuelles Triage eingeplant.

Regel 2 — `xml-dbf-validating-enabled`

```yaml
patterns:
  - pattern: $F.setValidating(true)
```

Feuert auf `setValidating(true)` → aktive DTD-Verarbeitung (XXE). Präziser als die Community-Regel: trifft den TP in `log4j-1.2-api`, ignoriert den gehärteten `log4j-core`.

Mechanismus: Pattern bindet jeden Aufruf an Metavariablen; `metavariable-regex` filtert per negativem Lookahead konstante String-Literale heraus. Keyword-basierte Sink-Discovery → hohe Trefferzahl, anschließend manuelles Triage.

Semgrep: Eigene Regeln, Ergebnisse und Triage

`custom_rules.yaml` ⟶ **15 Findings** (2 Regeln, 1488 Java-Dateien).

Regel 1 — 10 echte JNDI-Sinks (TP)
- `JndiLookup.java:63` ⟵ Log4Shell-Sink selbst
- `JndiManager.java:251`
- `JndiContextSelector.java:152`
- `JmsManager` (core + jakarta, 6×)
- `DataSourceConnectionSource.java:79`

Regel 1 — 4 Namens-Kollisionen (FP)

Methode heißt `lookup`, ist aber **kein** JNDI:
- `JavaLookup.java:70` — Log4j-`${java:}`-Lookup
- `MainMapResolver.java:80` — Map-Lookup
- `CommandLine.java:5392` — Enum-Lookup
- `FixedDateFormat.java:491` — Enum-Lookup

Regel 2 1 Treffer (TP): `XmlConfiguration.java:165` (`log4j-1.2-api`) isoliert; der gehärtete `log4j-core`-Parser wird **nicht** getroffen.

Kontrast: Community-Packs **0** JNDI-Treffer vs. eigene Regel **10** echte Sinks inkl. der Log4Shell-Stelle. Breite Sink-Discovery ⟶ manuelles Triage trennt die 4 Namens-Kollisionen sauber ab.

Gemeinsames Muster der False Positives

Die Regeln matchen **lokale Syntax** einen Algorithmus, einen API-Aufruf, ein Feature-Flag, einen Methodennamen ohne den **Verwendungs-Kontext**, der über echte Ausnutzbarkeit entscheidet:

Fehlender Kontext	Beispiel
Zweck der Operation	MD5 nur zur ID-Erzeugung, keine Krypto
Konfig-Absicht	Unverschlüsselter Socket ist gewollte Option
Äquivalente Härtung	Core-XXE anders abgesichert als erwartet
Access-Control-Guard	`if`-Guard schützt CI-Job, Regel ignoriert ihn
Namens-Kollision	`lookup()` ist nicht immer JNDI (Map/Enum)

Was Semgrep nicht fand: Generische Packs hätten auch **Log4Shell** (CVE-2021-44228) **nicht** erkannt ein **Design-Fehler** im JNDI-Lookup, kein lokales Muster. Selbst die eigene JNDI-Regel findet nur den **Sink**, nicht den vollständigen Datenfluss dafür ist Joern/Taint-Analyse nötig.

Vielen Dank für Ihre Aufmerksamkeit!

Wir sind offen für Fragen !!